Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчет
по практическому заданию №3

по дисциплине «Проектирование систем на кристалле»

Выполнила:

Алмаметова Ксения

P4130



Санкт-Петербург 2026

Аналогично, меняем rv32_isa_tests, чтобы был тест только на sub.

Что представляет собой тест на sub

1) Тест регистр-регистр (2 регистра-источника, 1 регистр-приемник)

```
TEST_RR_OP( 2,  sub, 0x0000000000000000, 0x0000000000000000, 0x0000000000000000 );
TEST_RR_OP( 3,  sub, 0x0000000000000000, 0x0000000000000001, 0x0000000000000001 );
```

```
#define TEST_RR_OP( testnum, inst, result, val1, val2 ) \
    TEST_CASE( testnum, x3, result, \
      li  x1, MASK_XLEN(val1); \
      li  x2, MASK_XLEN(val2); \
      inst x3, x1, x2; \
    )
```

```
#define MASK_XLEN(x) ((x) & ((1 << (__riscv_xlen - 1) << 1) - 1))

#define TEST_CASE( testnum, testreg, correctval, code... ) \
test_ ## testnum: \
    code; \
    li  x29, MASK_XLEN(correctval); \
    li  TESTNUM, testnum; \
    bne testreg, x29, fail;
```

По сути производится вычитание и сравнивается результат с ожидаемым результатом. Если не совпал, то прыгаем на fail (скрыт в макросе в конце теста в TEST_PASSFAIL) .
MASK_XLEN – подгоняет число под ширину регистра процессора, чтобы работало и на 32 и на 64 битном (оставляет только те биты, которые в регистр помещаются, а лишние старшие отбрасывает)

2) То же самое, но регистр-приёмник здесь совпадает со вторым регистром-источником.

```
TEST_RR_SRC1_EQ_DEST( 16, sub, 2, 13, 11 );
TEST_RR_SRC2_EQ_DEST( 17, sub, 3, 14, 11 );
```

```
#define TEST_RR_SRC1_EQ_DEST( testnum, inst, result, val1, val2 ) \
    TEST_CASE( testnum, x1, result, \
      li  x1, MASK_XLEN(val1); \
      li  x2, MASK_XLEN(val2); \
      inst x1, x1, x2; \
    )
```

В правильном процессоре инструкция должна сначала прочитать старое значение x2, а потом записать в него результат. В железе здесь не все очевидно, например, чтение и запись выпадают на один такт и возникает race condition, поэтому надо проверять.

3) Следующая пачка тестов
(TEST_RR_DEST_BYPASS/TEST_RR_SRC12_BYPASS/TEST_RR_SRC21_BYPASS)
проверяют, что процессор правильно разрешает RAW конфликт при разном расстоянии между инструкциями

3.1) Чтение результата sub после его записи

```
TEST_RR_DEST_BYPASS( 19, 0, sub, 2, 13, 11 );
TEST_RR_DEST_BYPASS( 20, 1, sub, 3, 14, 11 );
```

```
#define TEST_RR_DEST_BYPASS( testnum, nop_cycles, inst, result, val1, val2 ) \
    TEST_CASE( testnum, x6, result, \
      li  x4, 0; \
1:    li  x1, MASK_XLEN(val1); \
      li  x2, MASK_XLEN(val2); \
      inst x3, x1, x2; \
      TEST_INSERT_NOPS_ ## nop_cycles \
      addi  x6, x3, 0; \
      addi  x4, x4, 1; \
      li  x5, 2; \
      bne x4, x5, 1b \
    )
```

Самое главное здесь – надо чтобы addi прочитал x3 уже после того, как sub (inst) запишет туда результат

```
inst x3, x1, x2; \
```

```
TEST_INSERT_NOPS_ ## nop_cycles \
addi x6, x3, 0; \
```

3.2) Чтение операндов sub после их записи

```
TEST_RR_SRC12_BYPASS( 22, 0, 0, sub, 2, 13, 11 );
```

```
#define TEST_RR_SRC12_BYPASS( testnum, s
    TEST_CASE( testnum, x3, result, \
      li  x4, 0; \
1:    li  x1, MASK_XLEN(val1); \
      TEST_INSERT_NOPS_ ## src1_nops \
      li  x2, MASK_XLEN(val2); \
      TEST_INSERT_NOPS_ ## src2_nops \
      inst x3, x1, x2; \
      addi  x4, x4, 1; \
      li  x5, 2; \
      bne x4, x5, 1b \
    )
```

```
TEST_RR_SRC21_BYPASS( 28, 0, 0, sub, 2, 13, 11 );
```

```
#define TEST_RR_SRC21_BYPASS( testnum,
    TEST_CASE( testnum, x3, result, \
      li  x4, 0; \
1:    li  x2, MASK_XLEN(val2); \
      TEST_INSERT_NOPS_ ## src1_nops \
      li  x1, MASK_XLEN(val1); \
      TEST_INSERT_NOPS_ ## src2_nops \
      inst x3, x1, x2; \
      addi  x4, x4, 1; \
      li  x5, 2; \
      bne x4, x5, 1b \
    )
```

Отличаются разным порядком записи операндов

4) Проверка работы с регистром x0 (всегда равен 0, запись в него должна игнорироваться)

- x0 – первый операнд
- x0 – второй операнд
- x0 – и тот, и другой
- запись результата идет в x0 (должна игнорироваться)

```
TEST_RR_ZEROSRC1( 34, sub, 15, -15 );
TEST_RR_ZEROSRC2( 35, sub, 32, 32 );
TEST_RR_ZEROSRC12( 36, sub, 0 );
TEST_RR_ZERODEST( 37, sub, 16, 30 );
```

```
#define TEST_RR_ZEROSRC1( testnum, inst, result, val ) \
    TEST_CASE( testnum, x2, result, \
      li x1, MASK_XLEN(val); \
      inst x2, x0, x1; \
    )

#define TEST_RR_ZEROSRC2( testnum, inst, result, val ) \
    TEST_CASE( testnum, x2, result, \
      li x1, MASK_XLEN(val); \
      inst x2, x1, x0; \
    )

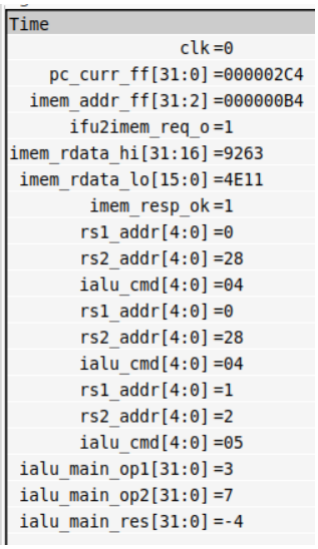
#define TEST_RR_ZEROSRC12( testnum, inst, result ) \
    TEST_CASE( testnum, x1, result, \
      inst x1, x0, x0; \
    )

#define TEST_RR_ZERODEST( testnum, inst, val1, val2 ) \
    TEST_CASE( testnum, x0, 0, \
      li x1, MASK_XLEN(val1); \
      li x2, MASK_XLEN(val2); \
      inst x0, x1, x2; \
    )
```

Вывод графиков

gtkwave build/simx.vcd (из обычного терминала , а не vs code)

На схему были добавлены



- clk
- PC – адрес инструкции, которая исполняется *сейчас* (на стадии EXU)
- imem_addr_ff - адрес, по которому IFU будет запрашивать *следующее* слово из памяти инструкций (запрашивается слово, а оно содержит 2 инструкции)
- ifu2imem_req_o – запрос к памяти инструкций прочитать слово
- imem_rdata_hi и lo – две половины слова, пришедшего из памяти инструкций (risc-v использует little endian)
- imem_resp_ok - внутренний сигнал IFU, что память ответила успешно, данные можно брать
- rs1_addr и rs2_addr – номера регистров, в которых лежат операнды (результат работы idu) (таких параметров на схеме несколько: зеленые – 2 конца одного и того же “провода”, который соединяет стадию idu с exu, поэтому значения там меняются одинаково, красные – уже на стадии exu, которая идет следующим тактом после idu)
- аналогично ialu_cmd – код команды, которую выоплняет ALU
- main_op и main_res – значения самих операндов и результат

Пример



один операнд вычитания лежит в регистре x1, другой – в x2, результат кладется в x3. Код инструкции sub = 05. Значения: 1-1 = 0

```
#define TEST_RR_OP( testnum, inst, result, val1, val2 ) \
    TEST_CASE( testnum, x3, result, \
      li  x1, MASK_XLEN(val1); \
      li  x2, MASK_XLEN(val2); \
      inst x3, x1, x2; \
    )
```

```
#define TEST_CASE( testnum, testreg, correctval, code...
test_ ## testnum: \
    code; \
    li  x29, MASK_XLEN(correctval); \
    li  TESTNUM, testnum; \
    bne testreg, x29, fail;
```

```
TEST_RR_OP( 3, sub, 0x0000000000000000, 0x0000000000000001, 0x0000000000000001 );
```

Потом надо сравнить результат с верным, т.е операция сравнения имеет код 09, сравнивается регистр x3 с x29

Аналогично будет 3-7=-4

```
TEST_RR_OP( 4, sub, 0xfffffffffffffffc, 0x0000000000000003, 0x0000000000000007 );
```

Добавляем Verilog модуль, который при обнаружении sub будет выводить регистры состояния

В верхний модуль тестбенча добавляем создание экземпляра модуля: пишем имя модуля, имя экземпляра и () означает подключение портов

```
src > tb > scr1_top_tb_ahb.sv
364   `endif // SCR1_IPIC_EN
390   );
391
392   scr1_tb_log_cmd i_scr1_tb_log_cmd();
393
394   endmodule : scr1_top_tb_ahb
395
```

Сам новый модель прописываем в отдельном файле

always_ff @(posedge scr1_top_tb_ahb.i_top.i_imem_ahb.clk) – на каждом фронте такта моста инструкций i_imem_ahb он выполняет свое тело – смотрит на каждую инструкцию, чтобы отловить sub

if (scr1_top_tb_ahb.i_top.i_imem_ahb.imem_resp == 2'b01) - если память подняла флаг OK, о том, что инструкция загружена корректно, то можно посмотреть на саму инструкцию, которая лежит теперь в imem_rdata. Это 32-битный регистр, там может лежать 32-битная инструкция (например, sub) / 16-битная сжатая (например, li) или 16-битная инструкция. Нам нужна 32-битная, у таких всегда в младших двух битах 11.

sub – это R тип инструкции. Такие инструкции имеют биты [6:0] = 0100000. В [14:12] пишется подгруппа операций, sub/add/mul относятся к 000. В [31:25] – уточнение собственно функции, для sub они равны 0100000

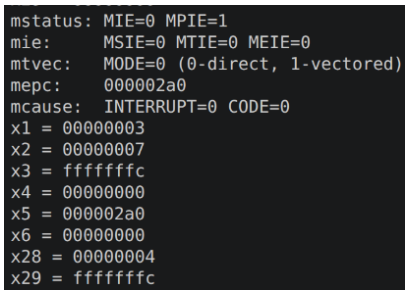
Вывод регистров

1) регситры CSR

```
grep -n "_ff" src/core/pipeline/scr1_pipe_csr.sv | grep -i "mstatus\|mepc\|mcause\|mtvec\|mie\|mip" | head -30
                csr_mstatus_mie_ff;     // MSTATUS: Global interrupt enable
                csr_mstatus_mpie_ff;    // MSTATUS: Global interrupt enable prior to the trap
                csr_mie_mtie_ff;        // MIE: Machine timer interrupt enable
                csr_mie_meie_ff;        // MIE: Machine external interrupt enable
                csr_mie_msie_ff;        // MIE: Machine software interrupt enable
                csr_mtvec_mode_ff;      // MTVEC: Mode (0-direct, 1-vectored) (registered)
                csr_mepc_ff;            // MEPC registered value
                csr_mcause_i_ff;        // MCAUSE: Interrupt
                csr_mcause_ec_ff;       // MCAUSE: Exception code
```

2) Регистры общего назначения (обычно все остальные нулевые, поэтому вывоже не все)

Пример вывода, как раз там где 3-7=-4 (пример с графика)



mstatus: MIE=0 — прерывания глобально запрещены, MPIE=1 - были ли прерывания разрешены до последней ловушки, нужно, чтобы после выхода из обработчика вернуть прерывания в прежнее состояние

mie – какие прерывания разрешены (запрещены все)

mtvec mode - прямой режим: все ловушки идут на один адрес

mepc - адрес начала теста (в конце _start mret записал в mepc _run_test)

mcause – прерываний не случалось, поэтому всё нулевое